

STELLAR: Storage Tuning Engine Leveraging LLM Autonomous Reasoning for High Performance Parallel File Systems

发表：2026/02/27

♥ 管理收藏夹

⚡ TL;DR 与摘要

🗺 思维导图

🖼 论文图片分析

🔍 相似论文推荐

PDF

原文

精读

内容目录

展开目录 ▼

1. 论文基本信息

1.1. 标题

论文全称为 *STELLAR: Storage Tuning Engine Leveraging LLM Autonomous Reasoning for High Performance Parallel File Systems*, 核心主题是提出了一套基于大语言模型自主推理的自动存储调优引擎, 专为高性能并行文件系统设计, 解决传统调优方法成本过高、效率低下的问题。

1.2. 作者与隶属机构

作者共5位, 分别是:

- Chris Egersdoerfer, 隶属美国特拉华大学
- Philip Carns, 隶属美国阿贡国家实验室
- Shane Snyder, 隶属美国阿贡国家实验室
- Robert Ross, 隶属美国阿贡国家实验室
- Dong Dai, 隶属美国特拉华大学 所有作者都是高性能计算 (HPC) 存储领域的专业研究者, 阿贡国家实验室是全球顶尖的HPC研究机构, 在并行文件系统领域有深厚的积累。

1.3. 发表渠道与影响力

本文目前发布在arXiv预印本平台, 尚未正式发表在期刊或会议, 属于大语言模型应用于复杂优化领域的最新前沿工作。

1.4. 发表时间

预印本发布时间为2026年2月26日（UTC），是领域内非常新的研究成果。

1.5. 摘要概括

研究目的：解决大规模并行存储系统调优复杂度高、人力成本高，传统自动调优方法需要大量迭代、探索成本过高，普通领域科学家难以使用的问题。核心方法：提出基于大语言模型智能体的自动调优框架STELLAR，模拟人类专家调优流程，整合检索增强生成（RAG）、工具调用、多智能体协作、知识积累机制，实现端到端自主调优。主要结果：STELLAR对从未见过的新应用，也能在前5次尝试内选出接近最优的参数配置，对比默认配置最高可实现7.8倍I/O性能加速，性能媲美甚至超过人类专家。关键结论：LLM智能体非常适合搜索空间大、探索成本高的复杂系统优化问题，能以极低的成本为领域科学家提供高质量的I/O调优结果。

1.6. 原文链接

预印本链接：<https://arxiv.org/abs/2602.23220v1> PDF链接：

<https://arxiv.org/pdf/2602.23220v1> 发布状态：arXiv预印本，尚未正式发表。

2. 整体概括

2.1. 研究背景与动机

- 核心问题：I/O性能是数据密集型科学计算的关键瓶颈，现代主流并行文件系统（如Lustre、Ceph）提供了成百上千个可配置参数，不同参数组合对不同应用的I/O性能影响差异极大，找到适配特定应用的最优参数组合是提升性能的核心。
- 现有挑战：
 1. 手动调优极度依赖HPC存储专家，人力成本极高，且系统默认配置是全局通用配置，无法适配不同应用的I/O模式，通常是次优结果。
 2. 传统自动调优方法（启发式规则、机器学习、强化学习）都需要成百上千次迭代探索搜索空间，每次迭代都需要运行应用测试性能，探索成本极高，在生产环境中完全不实用。
- 创新切入点：人类专家调优通常只需要几次尝试就能得到接近最优的结果，流程为分析I/O日志→基于知识选初始配置→测试反馈→调整配置→总结积累经验，效率远高于传统自动调优，而LLM智能体已经具备自主整合领域知识、推理决策、调用工具的能力，正好可以模拟人类专家的调优流程，解决传统方法的痛点。

2.2. 核心贡献/主要发现

核心贡献

1. 这是首个面向HPC并行文件系统的基于LLM的自动调优引擎，通过大量实验验证了该方案的有效性。
2. 提出了一套整合通用领域知识（来自系统手册）和集群特定知识（来自过往调优积累）的框架，实现了个位数迭代内的高效调优。
3. 设计了一套完整的LLM智能体工作流，支持和真实系统交互，通过迭代反馈提炼可复用的调优知识，持续提升未来调优效率。

主要发现

1. STELLAR可以在5次尝试以内得到接近最优的配置，对比默认配置最高获得7.8倍I/O加速，性能媲美甚至超过人类专家。
2. 积累的调优规则可以有效提升调优效率，不仅对同类型应用有效，也可以泛化到从未见过的新应用。
3. STELLAR的核心组件（RAG参数提取、分析智能体）都对最终性能有关键贡献，缺一不可。
4. 框架不依赖特定LLM，闭源大模型和开源70B参数模型都能获得不错的调优效果。

3. 预备知识与相关工作

3.1. 基础概念

- **并行文件系统（Parallel File System, PFS）**：专为高性能计算集群设计的分布式文件系统，将数据分散存储在多个存储节点，支持多个计算节点并行访问，提供高I/O带宽，是HPC系统的核心存储组件，常见代表为Lustre、Ceph。
- **参数调优（Parameter Tuning）**：并行文件系统提供大量可配置参数（如条带大小、条带数量、RPC并发数等），不同参数组合对不同I/O模式的应用性能影响极大，找到最优参数组合的过程就是调优。
- **大语言模型（Large Language Model, LLM）**：基于Transformer架构训练的大规模预训练语言模型，具备强大的自然语言理解、逻辑推理能力，支持调用外部工具和环境交互。
- **LLM智能体（LLM Agent）**：以LLM为核心控制单元的智能系统，能够感知环境、推理决策、调用工具自主完成复杂任务，不需要人工一步步指示。
- **检索增强生成（Retrieval-Augmented Generation, RAG）**：缓解LLM幻觉问题的常用技术，先将外部领域文档切分、嵌入建立向量索引，回答问题时先检索最相关的文档片段，再让LLM

基于准确的检索内容生成回答，大幅提升领域知识的准确性。

- **幻觉 (Hallucination)**：指LLM生成看似合理但事实错误的内容的现象，在领域特定任务中是严重问题。

3.2. 前人工作

3.2.1 并行文件系统自动调优

该领域的发展可以分为三个阶段：

1. **启发式规则方法**：代表工作如ASCAR、TAPP-IO，基于人工定义的规则针对不同I/O模式调优，缺点是规则覆盖不全，无法适配动态变化的 workload。
2. **传统机器学习方法**：代表工作如SAPPHIRE（基于贝叶斯优化），这类方法能从数据中学习调优策略，但仍然需要大量样本迭代。
3. **强化学习方法**：代表工作如CAPES、AIOC2，用深度强化学习学习调优策略，缺点是所有传统方法都需要成百上千次迭代测试，探索成本极高，在生产环境中无法实用。

3.2.2 LLM-based存储调优（数据库领域）

数据库调优和并行文件系统调优问题类似，最近也有LLM相关工作，代表如DB-BERT、GPTuner、E2ETune，这些方法将迭代次数从数千次降到了100次以内，但在HPC场景中，并行应用运行一次成本就很高，100次迭代仍然不可接受。同时数据库和并行文件系统的workload、架构、硬件环境差异极大，方法无法直接复用。

3.3. 技术演进脉络

存储自动调优的演进方向始终是降低探索成本、提升调优效率，路径为：人工调优→人工规则自动调优→传统机器学习/强化学习自动调优→LLM智能体自动调优。本文是首个将LLM智能体引入HPC并行文件系统调优的工作，将探索成本降低到个位数迭代，是该领域的重大突破。

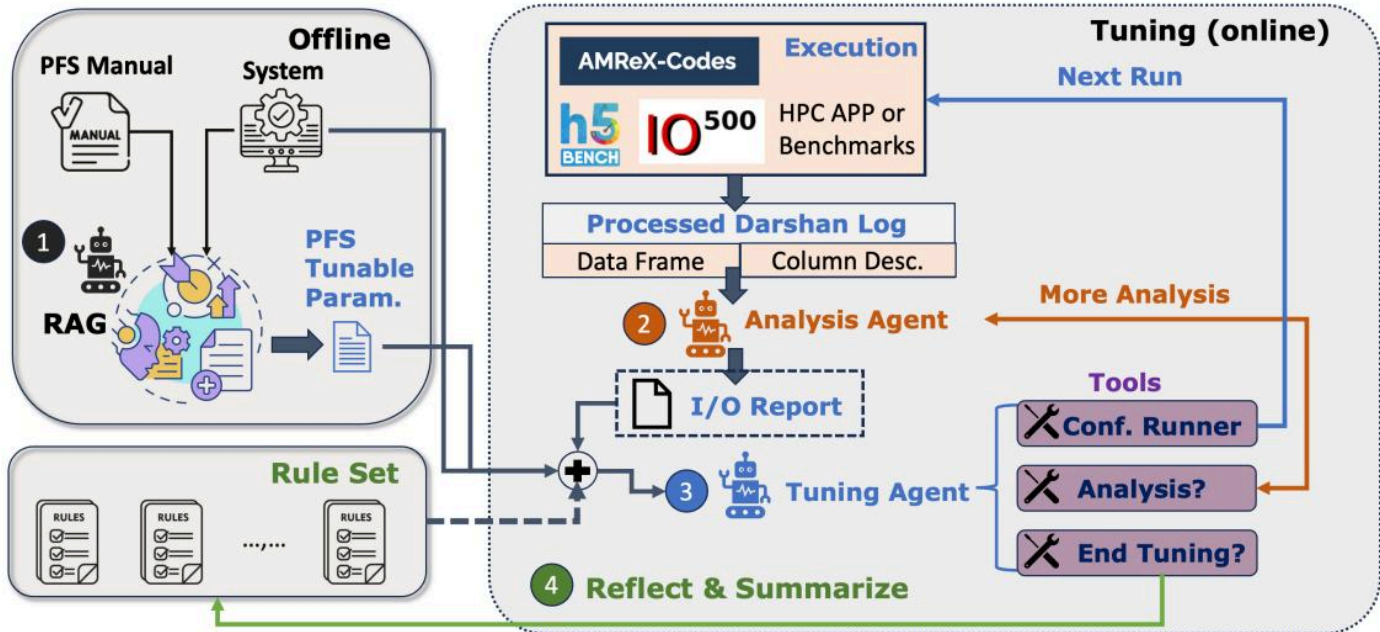
3.4. 差异化分析

- **对比传统自动调优**：STELLAR只需要最多5次迭代，效率比传统方法提升了几个数量级，完全适配生产环境的使用要求。
- **对比数据库领域的LLM调优方法**：STELLAR采用全自主智能体框架，覆盖日志分析、迭代反馈、知识积累全流程，将迭代次数降到5次以内，远低于数据库LLM方法的几十上百次，专门适配HPC场景高探索成本的特点。

4. 方法论

4.1. 整体架构与工作流程

STELLAR分为离线预处理和在线调优两个核心阶段，整体架构如下图（原文Figure 1）：



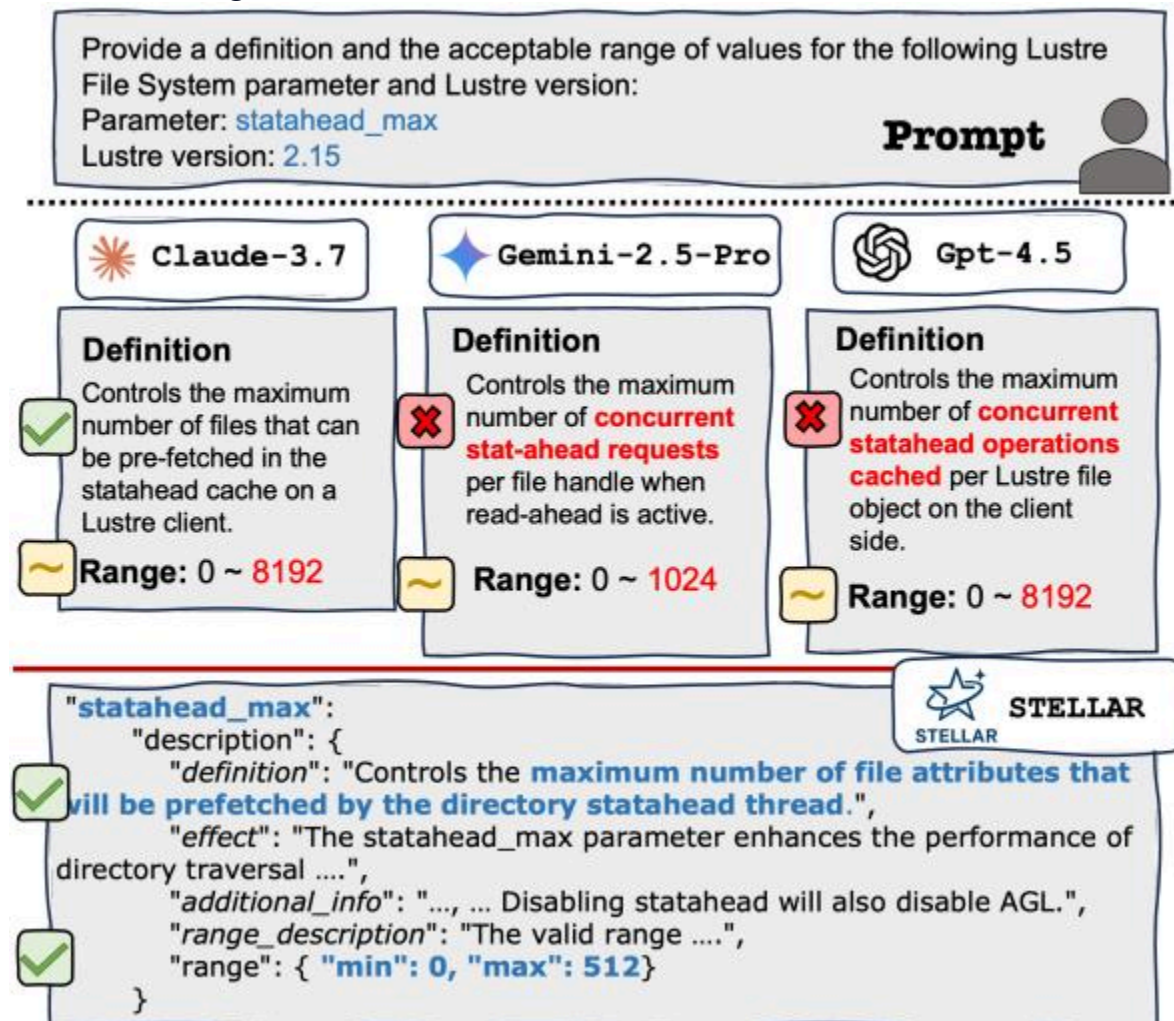
该图像是示意图，展示了STELLAR的设计概览。图中包含四个关键模块，分别为RAG提取参数、分析代理、I/O报告与调优配置，以及反思与总结。该系统通过离线和在线调优流程，整合了各种工具和分析方法，以优化高性能并行文件系统的参数配置。

完整工作流程：

- 离线阶段：** 从并行文件系统手册提取并筛选高影响可调优参数，解决LLM的幻觉和长文档处理问题。
- 在线调优阶段：**
 - 初始运行目标应用，生成I/O追踪日志（本文采用HPC领域常用的Darshan日志）。
 - 分析智能体分析日志，生成应用I/O行为报告。
 - 调优智能体基于参数信息、I/O报告、过往积累的调优规则，生成新参数配置，运行应用测试，收集性能反馈。
 - 重复迭代，直到调优智能体判断进一步调优无法获得明显性能提升，结束调优。
- 知识积累阶段：** 调优结束后，反思总结整个调优过程，提炼可复用的调优规则，加入全局规则库，用于未来的调优任务。

4.2. 基于RAG的参数提取模块

该模块的核心目标是解决LLM对领域特定参数的幻觉问题，作者通过实例验证，三个最先进的LLM对Lustre的`statahead_max`参数的描述都是错误的，而STELLAR的RAG方法可以得到正确结果，如下图（原文Figure 2）：



该图像是一个插图，展示了存储系统参数 `statahead_max` 的定义及其有效范围。比较了三个模型（Claude-3.7、Gemini-2.5-Pro 和 Gpt-4.5）对该参数的描述，指出其最大文件预取数值的不同，以及推荐的有效范围。

具体步骤：

4.2.1 建立向量索引

首先将并行文件系统的完整手册切分为1024词元（token）的块，块之间重叠20个词元，使用 OpenAI text-embedding-3-large 对每个块生成嵌入，建立可查询的向量索引。这种方式避免了将整部长文档放入LLM上下文，解决了长上下文信息丢失的问题，查询时只需要取出最相关的少数块即可。

4.2.2 参数识别与信息提取

1. 初始过滤：从系统接口（如Lustre的`/proc/fs/`和`/sys/fs/`目录）获取所有可写（可运行时修改）的参数，作为初始候选。

- 检索与生成：对每个候选参数，用查询语句How do I use the parameter [参数名]?检索向量索引，取出最相关的Top K个文档块，让LLM（默认GPT-4o）基于检索内容判断：文档是否包含足够信息描述参数作用、对I/O的影响、有效取值范围。如果信息足够，生成结构化的参数描述；如果不足，过滤该参数。
- 依赖参数处理：对于依赖其他参数或系统硬件的参数（如Lustre的max_read_ahead_per_file_mb最大值是max_read_ahead_mb的一半，后者最大值又依赖系统内存），要求LLM输出可解析的表达式，在线调优时自动根据实际系统计算合法范围。

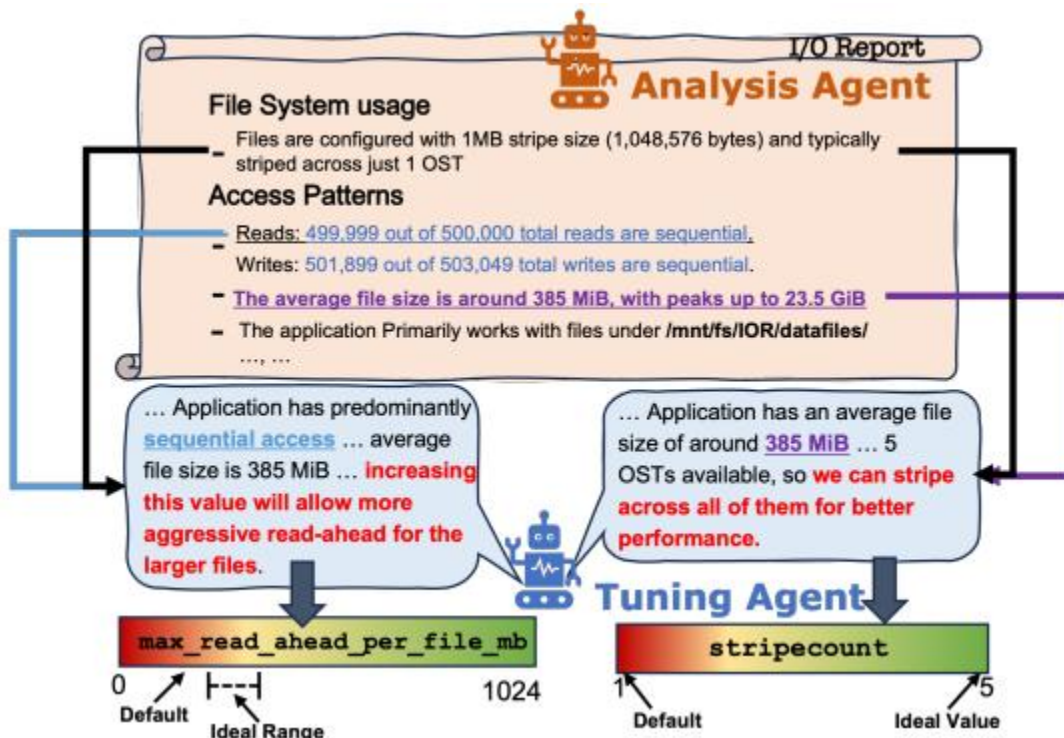
4.2.3 高影响参数筛选

STELLAR只保留对I/O性能有显著影响的参数，缩小搜索空间：

- 首先排除二值参数（如校验和开关），这类参数涉及用户需求权衡（数据完整性vs性能），不适合自动调优。
- 然后让LLM基于参数描述，判断参数是否对I/O性能有显著影响，只保留高影响参数。例如对Lustre，最终只筛选出13个高影响参数，大幅缩小了搜索空间。

4.3. 智能体在线调优

STELLAR采用两个协作的自主智能体完成在线调优，二者的交互决策示例如图（原文Figure 3）：



该图像是示意图，展示了分析代理与调优代理之间的决策互动。图中分析代理提供了文件系统使用情况和访问模式的报告，而调优代理则基于这些信息调整参数，如最大读取保障和条带数值。

4.3.1 分析智能体 (Analysis Agent)

核心作用：分析应用I/O追踪日志，生成I/O行为报告，还能响应调优智能体的需求完成定制化额外分析。实现：设计为可自主执行代码的LLM智能体，基于OpenInterpreter框架，能自主写代码、运行代码完成分析任务。输入：预处理后的Darshan日志（转换为带列描述的Pandas DataFrame格式）。输出：应用I/O行为的高层总结，包括文件大小分布、访问模式、读写比例、元数据操作比例等调优关键信息。如果调优智能体认为信息不全，会动态要求额外分析，灵活适配不同应用的需求。

4.3.2 调优智能体（Tuning Agent）

核心作用：控制整个迭代调优过程，生成参数配置，收集性能反馈，判断调优终止时机。初始输入：筛选后的高影响参数信息、集群硬件存储信息、分析智能体生成的I/O报告、过往积累的全局调优规则库。调优智能体可以调用三类工具完成决策：

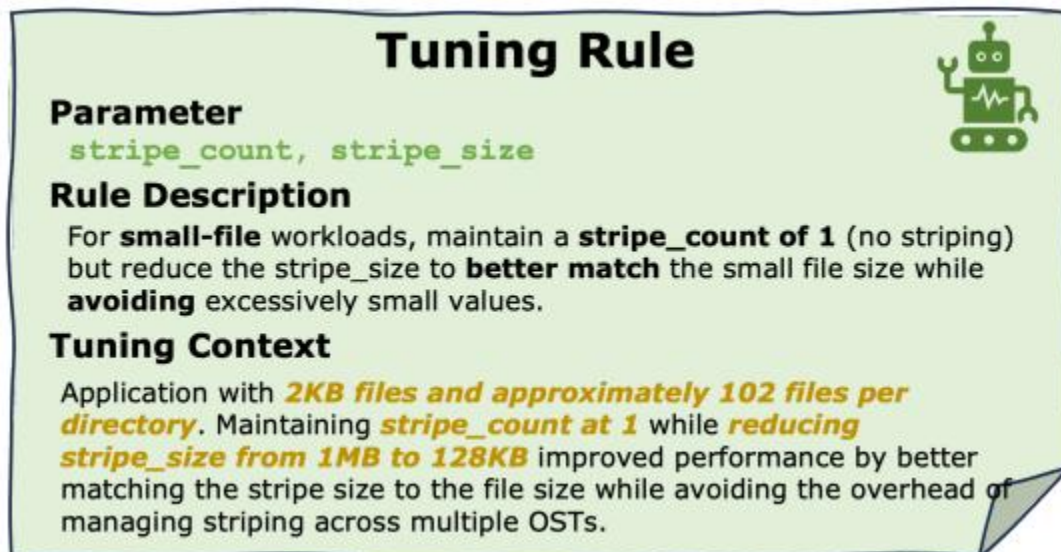
1. **分析工具**：如果缺少关键信息，调用该工具向分析智能体发起特定分析请求，获取额外信息。
2. **配置运行工具**：信息足够时生成新参数配置，**显式记录每个参数取值的理由**，然后驱动应用用新配置重新运行，收集性能数据作为反馈。记录理由是为了后续总结调优规则做准备。
3. **结束调优工具**：当判断进一步调优不会带来明显性能提升时，调用该工具终止调优循环，触发反思总结。终止规则：当性能提升进入收益递减阶段就停止，控制探索成本。

4.3.3 设计选择：为什么用自主智能体而非固定工作流？

不同应用的I/O复杂度差异大，调优需要的步骤也不同，自主智能体可以动态延长或缩短调优过程，动态调整分析重点，比固定工作流更灵活，能适配不同复杂度的应用。

4.4 调优规则积累模块

调优结束后，STELLAR会将本次调优经验提炼为可复用的调优规则，加入全局规则库，供未来调优新应用使用，示例如图（原文Figure 4）：



该图像是图示，展示了针对小文件工作负载的调优规则，具体参数包括 `stripe_count` 和 `stripe_size`。规则建议在保持 `stripe_count` 为 1 的情况下，调整 `stripe_size` 以匹配小文件的大小，避免过小的值。应用示例中，2KB 文件和每个目录约 102 个文件的情况下，通过将 `stripe_size` 从 1MB 降至 128KB，提高了性能。

4.4.1 规则结构设计

每个规则都是结构化JSON对象，包含三个核心字段：

1. `Parameter`：规则针对的参数名称。
2. `Rule Description`：调优规则的通用描述，不绑定特定应用，只给出指导性建议，例如针对元数据密集型小文件负载，`stripe_size` 应匹配文件大小，避免设置过小。
3. `Tuning Context`：规则适用的I/O场景描述，例如每个目录包含上百个2KB–8KB小文件，以元数据操作为主，方便未来匹配相似I/O模式的新应用。

4.4.2 规则合成与冲突解决

新调优结束后，将新规则和现有全局规则库合并，按以下规则解决冲突：

1. 如果新规则和现有规则在同一参数、同一场景下给出完全相反的指导，删除两个规则，因为无法判断正确性。
2. 如果只是指导略有不同，保留为两个可选方案，未来调优中验证，保留有效的，删除无效的。通过这种方式，规则库会随着调优次数增加越来越准确，越来越有用。

5. 实验设置

5.1 实验平台

所有实验在开放平台CloudLab完成，搭建了10节点测试集群：

- 每个节点配置：Intel Xeon Silver 4114处理器（10物理核）、196GB内存、10Gbps网络。
- 软件配置：部署Lustre 2.15.5并行文件系统，5个节点做对象存储服务器，1个节点做管理+元数据服务器，剩余5个做客户端节点运行应用。
- 所有应用都使用50个MPI进程，分布在5个客户端节点运行。

5.2 测试数据集/workload

测试分为基准测试和真实应用两类，覆盖不同I/O模式：

5.2.1 基准测试程序

1. **IOR**：经典I/O性能基准，两个workload：

- IOR_{64K} ：64KB传输块，随机访问共享文件，代表随机小块I/O。
- IOR_{16M} ：16MB传输块，顺序访问共享文件，代表顺序大块I/O。

2. **MDWorkbench**：元数据性能基准，两个workload：

- $MDWorkbench_2K$ ：每个进程创建10个目录，每个目录400个2KB文件，代表小文件元数据密集负载。
 - $MDWorkbench_8K$ ：类似，每个文件大小8KB。
3. **IO500**：官方综合基准，混合了大块顺序I/O、小块随机I/O、空文件元数据、小文件元数据，代表混合负载。

5.2.2 真实应用

1. **AMReX**：块结构自适应网格细化科学计算框架的I/O核心，是真实科学应用的代表。
2. **MACSio**：多物理场应用I/O代理，两个配置： $MACSio_512K$ （对象大小512KB）、 $MACSio_{16MB}$ （对象大小16MB）。

5.3 评估指标

本研究的核心评估指标为：

1. **运行时间 (Wall Time)**：完成应用整个I/O过程的实际时间，单位为秒，越小表示性能越好。
2. **加速比 (Speedup)**：默认配置运行时间除以调优后配置运行时间，即：

$$\text{Speedup} = \frac{T_{\text{default}}}{T_{\text{tuned}}}$$

其中 T_{default} 是默认配置的运行时间， T_{tuned} 是调优后配置的运行时间。加速比越大，调优效果越好，加速比大于1表示调优后性能优于默认配置。

3. **迭代次数 (Number of Attempts)**：达到收敛需要的尝试次数，越少表示调优效率越高，本研究限定最多5次尝试。

5.4 对比基线

- **默认配置基线：**并行文件系统的出厂默认参数配置，代表普通用户不做调优的基准性能。
- **人类专家基线：**给人类专家提供完整的workload信息和Darshan日志，不限时间给出最优配置，代表人类专家调优的上限。

6. 实验结果与分析

6.1 核心结果分析（从零开始调优）

STELLAR从零开始（无任何积累规则）调优，限制最多5次尝试，结果如下图（原文Figure 5）：



该图像是一个比较不同存储调优方法下的运行时间条形图。图中左侧为 IOR 的不同配置，右侧为 MDWorkbench 和 IO500，显示了 default、expert 和 STELLAR 方法的耗时差异，STELLAR 在大多数情况下明显表现出更优的性能。

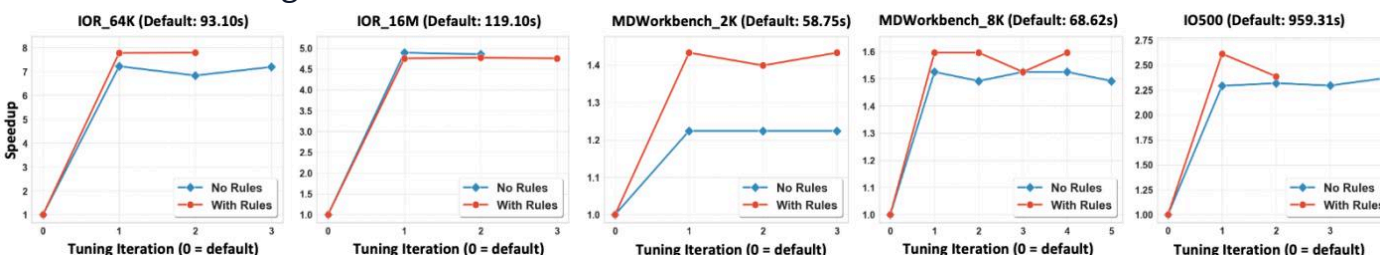
图中纵轴为运行时间，越小越好：

- 对比默认配置：STELLAR对所有workload都获得了显著性能提升，最高达到7.8倍加速，验证了方法的有效性。
- 对比人类专家：STELLAR的性能和人类专家非常接近，甚至在IO500混合负载上性能超过了人类专家。结论：STELLAR可以在5次尝试以内得到接近最优的配置，性能媲美人类专家。

6.2 积累规则后的调优性能

6.2.1 规则插值（对已调优的基准，用规则库重新调优）

结果如下图（原文Figure 6），横轴为迭代次数，纵轴为加速比，越大越好：



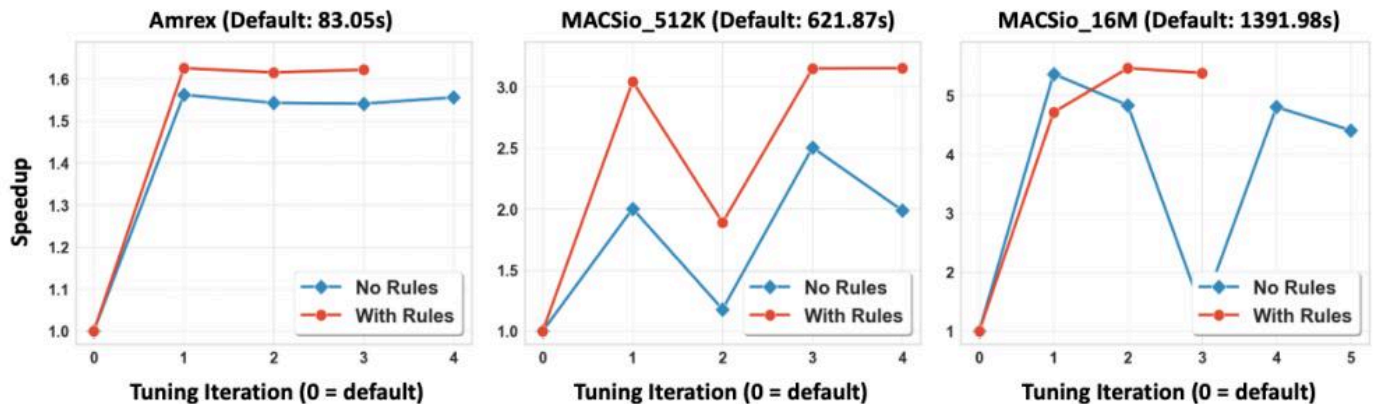
该图像是一个图表，展示了不同调整规则对多个测试程序速度提升的影响，包括 IOR、MDWorkbench 和 IO500 等。每个子图呈现了在不同调整迭代下的速度提升效果，分别以没有规则和有规则两条线表示。

结果：

- 5个基准中的4个，第一次尝试就能获得显著更高的加速比，因为规则库帮助调优智能体选对了初始方向。
- 初始性能提升后，更少的迭代就能收敛，3个基准需要的迭代次数减少，1个持平，所有都在5次以内收敛。
- 原本STELLAR在MDWorkbench_2K上略差于人类专家，加入规则库后性能追上了人类专家。证明规则积累有效提升了调优性能和效率。

6.2.2 规则外推（对未见过的真实应用，只用基准积累的规则库调优）

结果如下图（原文Figure 7），横轴迭代次数，纵轴加速比：



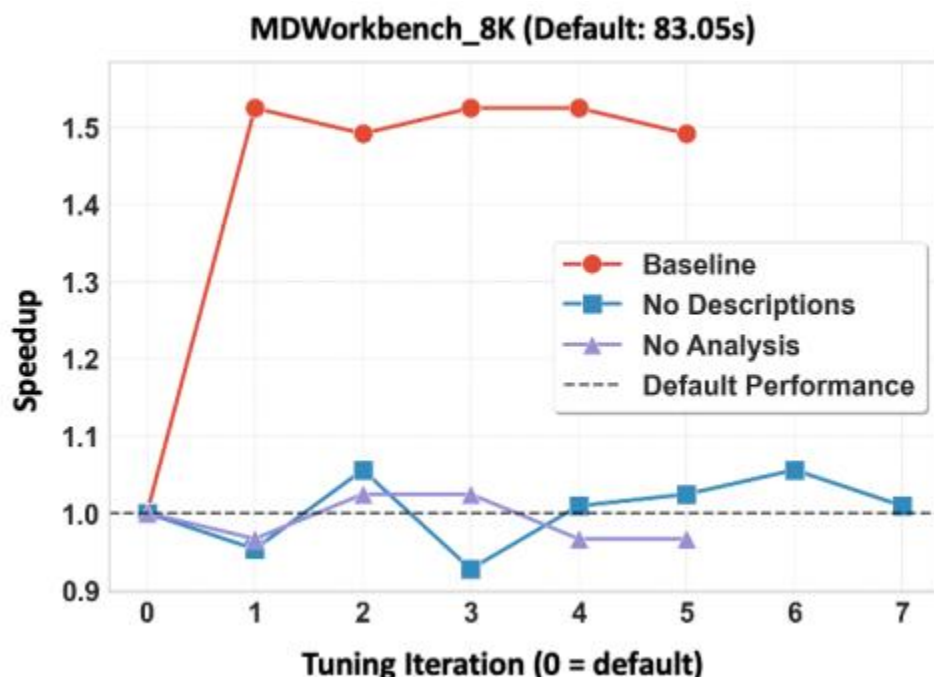
该图像是性能加速对比图，展示了在不同调优迭代下，使用规则和不使用规则的三种实例（Amrex、MACSio_512K和MACSio_16M）的速度提升。图中显示，使用规则的参数调整在多次迭代中显著提高了性能表现。

结果：

- 哪怕是从未见过的真实应用，积累的规则库也能显著提升调优性能，让收敛更稳定，每次尝试的平均性能更高。
- 规则库帮助STELLAR避开了差的配置：例如MACSio_16M无规则时第一次尝试性能和默认相当，有规则后第一次就获得高加速比；MACSio_512K无规则时会探索到很差的配置，有规则后成功避开。结论：积累的调优规则可以有效泛化到未见过的新应用，提升调优效率。

6.3 消融实验（组件有效性验证）

对最难调优的MDWorkbench_8K做消融实验，结果如下图（原文Figure 8）：

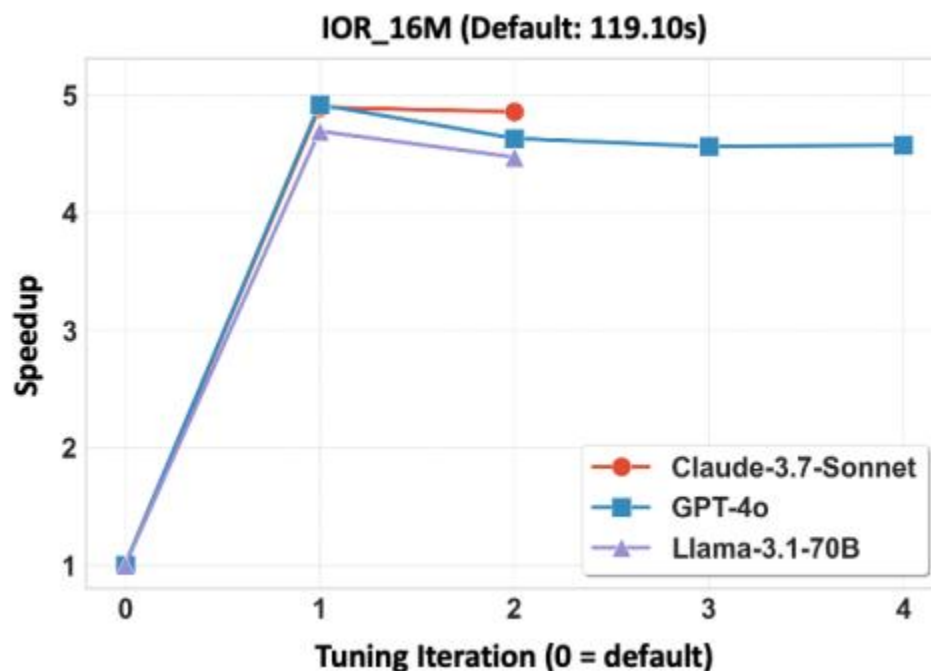


该图像是一个图表，展示了在 MDWorkbench 中调优性能的不同尝试结果。横轴为调优迭代次数，纵轴为加速比。红色圆点表示基准线，蓝色方块表示无描述情况，紫色三角形表示无分析情况，虚线表示默认性能。

- **No Descriptions:** 移除RAG生成的参数描述，只保留合法取值范围。结果性能大幅下降，原因是调优智能体不理解参数的真实含义，会做出错误决策，例如错误理解条带数量的作用，选错参数。证明RAG参数提取模块是性能的关键保障。
- **No Analysis:** 移除整个分析智能体，没有I/O行为分析。结果性能下降更严重，几乎和默认配置相当，原因是调优智能体不知道应用的I/O模式，选错调优方向，例如对小文件密集负载还去增大预读和RPC大小，完全走错方向。证明分析智能体是正确调优的基础。结论：STELLAR的两个核心组件都不可或缺，对最终性能都有关键贡献。

6.4 不同LLM的性能对比

测试了三个不同LLM作为调优智能体在IOR_16M上的性能，结果如下图（原文Figure 9）：



该图像是一张图表，展示了不同大语言模型（LLM）作为调优代理在 IOR_16M 上的调优性能。X 轴表示调优迭代次数（0 = 默认），Y 轴表示加速比。图中的三条曲线分别对应于 Claude-3.7-Sonnet、GPT-4o 和 Llama-3.1-70B。调优过程显示，使用 LLM 的调优策略在初始迭代后显著提升了性能。

三个 LLM 分别是闭源的 Claude-3.7-Sonnet、GPT-4o，以及开源的 Llama-3.1-70B-Instruct，结果显示：三个 LLM 都能获得接近的性能，都能得到近 5 倍的加速比，远高于默认配置。证明 STELLAR 框架不依赖特定的 LLM，任何支持工具调用的 LLM 都可以使用，哪怕是 70B 参数的开源模型也能获得不错的效果。

6.5 成本分析

- **延迟开销**：LLM 推理每次决策只需要几秒钟，相对于 HPC 应用几分钟到几小时的运行时间，开销可以忽略不计，端到端调优时间主要由应用运行时间决定。
- **Token 成本**：一次完整调优平均，Claude-3.7-Sonnet 约 100k 输入 token、13k 输出 token；GPT-4o 作为分析智能体约 400k 输入 token、8k 输出 token。Prompt 缓存可以复用 85%-90% 的输入 token，大幅降低了成本，无论是 API 调用还是本地部署都非常经济。

7. 总结与思考

7.1 结论总结

本文提出了首个基于 LLM 自主智能体的并行文件系统自动调优引擎 STELLAR，通过模拟人类专家的调优流程，整合 RAG、多智能体协作、工具调用和知识积累机制，解决了传统自动调优探索成本过

高的核心问题。实验证明，STELLAR哪怕对从未见过的新应用，也能在最多5次尝试内得到接近甚至超过人类专家的调优结果，积累的知识可以持续提升未来调优的效率，为搜索空间大、探索成本高的复杂系统优化提供了新的可行方向，能够让普通领域科学家以极低的成本获得高性能I/O配置，加速科学发现。

7.2 局限性与未来工作

作者提出的局限性和未来方向：

1. **规模限制：**目前实验都在小规模CloudLab集群完成，没有在大规模生产HPC系统验证，主要原因是并行文件系统参数调优需要root权限，生产环境通常不开放。未来工作将聚焦于用户可访问的调优参数，包括应用层参数（HDF5、MPI-IO参数）、不需要root权限的用户态存储系统（如DAOS），同时为管理员提供系统级参数的调优推荐。
2. 作者认为STELLAR的核心方法是规模不变的，无论系统大小，都能模拟人类专家的调优流程，未来将在大规模生产系统完成进一步验证。

7.3 个人启发与批判

启发

1. 该工作很好地展示了LLM智能体在复杂系统优化领域的独特优势，对于需要领域知识、探索成本高的问题，模拟人类专家思路的LLM智能体比传统黑盒机器学习方法效率高很多，这个思路可以推广到很多其他系统优化问题，比如集群调度、网络配置、编译器优化等，是非常有前景的方向。
2. 知识积累复用的设计非常巧妙，让STELLAR越用越好用，符合人类专家能力增长的规律，这是传统自动调优方法不具备的特点。
3. 对LLM幻觉的处理思路工程性很强：用RAG解决参数知识的幻觉，用真实系统反馈纠正调优决策错误，用多智能体分工降低错误概率，整套方案非常实用，适合落地。

潜在可改进的点

1. 目前实验都在小规模集群完成，还需要大规模生产系统的验证，尤其是大规模系统中参数交互更复杂，需要验证STELLAR的性能是否保持稳定。
2. 目前调优终止由LLM自主判断，未来可以加入更量化的终止标准，比如连续两次性能提升低于阈值就停止，进一步控制调优成本。
3. 目前规则冲突解决还比较简单，未来可以引入权重机制，根据规则过往生效概率给不同规则分配不同权重，帮助调优更快找到正确方向。

